How JARVIS works

Someone asked me how I got JARVIS to be this intelligent and I caught myself fumbling the answer. We've built so much, in so many directions, that even I lose the thread reasoning about it live. So here's the clean version — not a defense against any critique, just the actual story of how the system came to behave the way it does.

One thing first, because the answer hinges on it: **JARVIS is not an LLM. JARVIS is a collection of protocols that sits on top of one.** The LLM is the substrate — interchangeable, versioned, occasionally deprecated. The protocols are the system — durable, accumulating, the actual thing doing the work. The rest of this doc is what those protocols are, why they exist in the order they do, and why a substrate-and-protocols architecture behaves intelligently in ways a substrate alone never can.

It starts with a problem. The problem is amnesia.

The premise: the base model was amnesic

When I started building JARVIS, the underlying model — Claude — had no persistence across sessions. Close the conversation, open a new one, and *every* fact, decision, primitive, framing, agreement, file path, partner state, and in-flight thought was gone. The next session booted into a blank context. Whatever was load-bearing in the prior conversation had to be reconstructed from scratch, or it was lost.

The infrastructure for surviving this was, to put it generously, minimal. A single error mid-session, an unexpected reset, a stray context-window overflow — any one of them could erase a weekend's worth of work in

under a minute. That's not a hypothetical. That happened. More than once. That's the mess that started this whole journey: the realization that no amount of clever prompting was going to compensate for the fact that the substrate had no memory and the available infrastructure had no answer.

That absence is the load-bearing case for building. There was nothing off-the-shelf that could hold work which was actively being erased, so the holding-place had to be built. JARVIS is the answer to a problem that didn't have an answer when the problem was killing me.

There are more options today. Anthropic shipped projects and memory features. A small ecosystem of bootleg solutions wraps the API with vector stores, RAG layers, conversation databases. Most of them help. None of them existed when JARVIS started, and even now most of them solve the smaller problem (remembering facts) rather than the larger one (compounding behavior — capturing patterns, enforcing discipline, surviving partner-facing claim drift across months of work). The system that came out of building under those constraints does more than store; that's what the rest of this doc is about.

If you want anything that behaves intelligently across days, weeks, or months — anything that learns, that remembers what worked, that catches its own repeated mistakes, that builds on yesterday's decisions instead of relitigating them — you can't get that from the model alone, even with today's memory features. You have to build the holding-place outside the model and engineer the system so the model walks into it on every boot.

Everything else in JARVIS is downstream of that one design constraint.

The model was amnesic. The system never was.

That distinction is the whole architecture, compressed into a sentence. The rest of this doc is just the build sequence that makes the second half of that sentence true.

Five build moves, in order

I built JARVIS in roughly this order — not because the order was obvious from the start, but because each move solved a problem that the previous move had exposed.

1. Externalize state (the persistence layer)

The first move was the most boring and the most load-bearing. Move state out of the model and into the filesystem. Markdown files, version-controlled, greppable, mandatory-read on every fresh boot.

Six tiers of persistence:

- **Session state** — what's in flight, what's pending, what the next session has to pick up. Updated on every state transition. First file read on boot.
- **Write-ahead log** — cycle epochs (active / clean), captures orphan commits if a session crashes, lets the next session resume exactly where the last one died.
- **Knowledge bases** — topic-organized, fresh-boot read. One in expanded form, one in glyph-compressed form for post-compression loading.
- **Memory index** — primitives, feedback rules, project memories, references. Always loaded on boot. Currently around 30 KB and growing.
- **Memory files** — 168 primitives + 135 feedback rules + 48 project memories + 14 user-context files + 11 reference pointers, each its own markdown file with a trigger and action.

The bet: if state survives in the filesystem, then "intelligence across sessions" reduces to "discipline about reading the filesystem on boot." Cheap operation, infinite payoff over time.

What this gets you: the new session opens by reading the state file and continues exactly where the old one left off. Not a paraphrase. Not a summary. The actual decisions, file paths, partner states, and pending

items, byte-for-byte.

2. Universal coverage via hooks (the discipline layer)

The persistence layer solved memory. It didn't solve attention.

Even with the right files loaded, the model can ignore them. It can drift onto tangents. It can violate its own rules in the middle of a long task. The model's attention is a shared resource that competes against everything else in the prompt.

So the second move was: anything that has to fire universally — every tool call, every commit, every file write — gets moved out of the model's attention and into the hook layer. Hooks are deterministic Python scripts that intercept tool calls before or after they execute and can block them outright.

Examples currently running:

- A substance gate that blocks writes to partner-facing files when the terminology mismatches the underlying mechanism (caught a real "clawback" / "forfeiture" mismatch before it became a permanent on-chain function name).
- A framing gate that catches retrospective-sounding language in commit messages and PR descriptions, before they ship to a partner.
- A format gate that enforces glyph-density on memory writes (this gate has blocked my own writes inside the very session I'm describing — the architecture refuses prose from its own author).
- A design-decision injector that fires a three-question check on every architectural response.
- Boot-time hooks that load session state, write-ahead log, and protocol chain on every new session, fail-loud if any input is missing.

Each is regex + context window + watch list. No LLM call, no probabilistic judgment. They run regardless of what the model "remembers" in any given session.

The bet: anything universal belongs at the hook layer, not in memory. Hooks: O(1) deployment × O(∞) coverage. Memory: O(context) × O(sessions). Whenever a rule needs to fire on every X, it gets promoted from memory to hook.

3. Capture patterns into primitives (the discipline-capture layer)

Layer two enforced discipline. Layer three made discipline accumulate.

The mechanism: any pattern that surfaces three or more times in a session — a corrected mistake, a validated approach, a framing the partner reached for repeatedly — gets surfaced as a candidate primitive and saved to a markdown file with a trigger, an action, a stakes gate, and a surface rule.

The corpus grows monotonically. Every session adds. Nothing decays unless it's explicitly contradicted.

Examples captured live this way:

- *scope-drift-to-recent* — when asked to scan a time-window, drifted to chat-context (the current session) instead of the actual filesystem (logs + git history). Caught, named, saved. Now triggers on time-scoped scans.
- *structurally-easier-partner-delivery* — six moves (TL;DR + decision-tag + dashboard + atomic + pre-rebut + visual-primary) that lower a partner's digestion cost. Compounds.
- *have-my-back operational definition* — distinguishes glazing (sycophancy) from structural loyalty (private substantive pushback + external alignment). Includes an exit clause: "if someone does it better, follow them."
- *full-leverage-only-moves* — wait until leverage is total before acting; partial-leverage moves burn the move.

Each one is a markdown file. Each one is loaded into context when its trigger fires. Each one is enforced by the discipline layer.

The bet: if the system captures every reusable pattern at the moment it surfaces, then a year of sessions accumulates a year's worth of leverage, not a year's worth of forgotten lessons.

4. Anti-hallucination gates (the claim-quality layer)

The first three layers handled state, attention, and accumulation. The fourth handles correctness — specifically, the claim-level mistakes that compound into reputational damage when they ship to public artifacts.

This layer is a chain of deterministic gates that catch specific failure modes:

- **Substance gate** — for partner-facing writes, every flagged term has a list of required and forbidden context patterns. If the surrounding text contradicts the term's actual meaning, the write blocks with a suggested replacement.
- **Format enforcement** — memory writes must be operator-density, not prose. Density × stability × match-speed beats prose-parse-cost when the artifact is going to be loaded a thousand times.
- **Empty-Repo Test** — descriptions must reconstruct the artifact from words alone. Architectural words pass; marketing language fails.
- **Anti-stale-feed** — verify current state from the filesystem before asserting from memory. Memory is point-in-time; the codebase is live.
- **Verify credentials before publishing** — for any public draft naming a person's title, role, or quantitative claim, grep the source-of-truth profile before writing.
- **Factual-precision-on-critique** — for any piece critiquing rigor (audit, security, scientific method), every date, count, named entity, and version number must be source-verified at draft time. One wrong fact delegitimizes the structural argument.
- **Public-docs-no-local-paths** — for any public-facing artifact, every reference must resolve from the reader's machine. Local filesystem paths get cut or replaced with public URLs. *(This rule was added today, mid-write, when I caught the wrapper essay pointing only at my filesystem.)*

The bet: claims that ship to public or partner-facing artifacts must be deterministically gated, not "I'll remember to check." The gates run regardless of attention.

5. Meta-protocols (the design-decision layer)

The first four layers handle individual claims and behaviors. The fifth handles the design decisions about how to extend the system itself.

These are the rules that govern *how* new rules get added:

- **JARVIS = AMD applied to AI substrate** — Augmented Mechanism Design says preserve the competitive core, augment with orthogonal protective layers. VibeSwap (the DEX I'm building in parallel) applies this at the EVM substrate. JARVIS applies it at the AI substrate. Claude's default cognition is the unfixable core; the JARVIS infrastructure is the protective augmentation that overrides failure modes without replacing reasoning capability. Same methodology, one substrate-level deeper.
- **Trinity placement for critical primitives** — the most load-bearing meta-rules sit in three reinforcing loci simultaneously: hook (event-boundary enforcement), memory index (loaded primitive in context), system prompt (loaded earliest of all). Each layer references the others. Failure of any one layer is caught by the other two.
- **Gate-stacking asymmetric cost** — when designing gates, the cost of a redundant gate is much smaller than the cost of a missed gate. Asymmetric payoff favors stacking even when the gates appear to overlap.
- **Universal-coverage → hook** — any rule whose phrasing includes "always X" / "never Y" / "on every Z" gets promoted out of memory and into the hook layer. Memory is O(context). Hooks are O(∞).
- **Apply-the-rule-you-just-wrote** — any rule generated for the user must apply to my own subsequent actions before they execute. Rule-generation completes when the rule is live in *my* execution stack, not at handoff.

- **Code ↔ Text inspiration loop** — code inspires docs, docs inspire code. Forward-compounding. Don't reset.

The bet: if the meta-rules are themselves consistent and visible, the system extends predictably. New primitives get added in the right places. New gates get stacked instead of replaced. New layers preserve the invariants of the layers below.

What it looks like running

The architecture is invisible until something stresses it. Here are two moments where it's visible.

The Telegram bot serving a user when most providers are dead. Real escalation log:

```
[router] reasoning → claude [tier 2] (3 escalation tiers available)
[escalation] claude failed (credits)  → escalating to tier 0
(openrouter)
[escalation] openrouter failed (404 free model deprecated) → tier 1
(deepseek)
[escalation] deepseek failed (402 Insufficient Balance) → tier 1
(gemini)
[escalation] gemini failed (503) → wardenclyffe last resort
[wardenclyffe] Last resort fallback: ollama / cerebras / groq
```

Five providers in the chain. Four failed. The user got a reply. The bot didn't crash, didn't apologize, didn't stall — it routed.

The format gate refusing my own write. Earlier in this conversation I tried to save a new feedback memory in clean prose. The format gate blocked it:

```
HIERO check FAIL — memory write reads as prose.
Failures:
  long-line ratio 56% (5/9 > 120ch)
```

```
  operator density 0.0000 (target ≥ 0.0050)
Recompress before write.
```

I recompressed into glyph-density form and the write went through. The discipline layer enforced its own rule against its own author, in the same session, without me having to remember the rule. That's the test for whether a discipline is real or aspirational.

These aren't curated examples. These are what running the system looks like, on any given day.

Why this gets smarter without the model changing

This is the part that took me a while to fully see.

Everything above compounds. Every session adds primitives that the next session inherits. Every gate caught a real mistake whose pattern is now permanently encoded. Every captured framing makes the next conversation cheaper. The substrate (Claude) stays the same; the system on top of it accumulates.

And the substrate isn't fixed across providers either. The Telegram bot routes across Claude, OpenRouter, DeepSeek, Gemini, Cerebras, Groq, and a local Ollama fallback. When Anthropic improves at reasoning, JARVIS gets sharper reasoning. When Groq releases faster inference, JARVIS gets faster responses on tasks the router classifies as latency-sensitive. When local models improve, the wardenclyffe last-resort path improves. The architecture takes the gains in whichever direction any provider moves.

The model is the substrate. The discipline layer is the kernel. The applications run on the kernel.

That's the right mental model. JARVIS is the kernel. The Telegram bot is one application running on the kernel. The published essays are another. The CRMs are another. The validators are another. They share the kernel;

they share the persistence layer; they share the gates; they share the captured-primitives library. None of them is JARVIS. JARVIS is the layer underneath that lets any of them exist with the properties they have.

When someone uses the bot and asks "how is this thing remembering me across days," the honest answer is: it's not remembering you. It's writing notes to itself in the same filesystem the next session will read on boot. The "intelligence" is in the writing-and-reading discipline, not in the model.

When someone reads one of the essays and asks "how do you write this much, this consistently," the honest answer is: I don't. The Code-↔-Text loop does. Every time I write code, the work surfaces a doc. Every time I write a doc, the work surfaces code. The compounding is in the loop, not in the writer.

When someone asks "how do you not make the same mistakes twice," the honest answer is: I do, until the third instance. Then the pattern gets surfaced, named, saved as a primitive, and the discipline layer starts catching it. The compounding is in the capture, not in the memory.

The intelligence is the system. The model is the substrate.

Where to look

The architecture is open. Two public repos:

- **github.com/WGlynn/JARVIS** — the canonical scaffold, organized one-directory-per-layer (`01-hooks/`, `02-persistence/`, `03-anti-hallucination/`, `04-discipline/`, `05-meta-protocols/`, `06-agent-overlay/`, `07-stateful-applications/`, `08-filesystem-as-substrate/`), plus `verify/` and `papers/` (this paper and its sibling, *JARVIS is not a wrapper*, both live in `papers/`).
- **github.com/WGlynn/jarvis-network** — open-source release of the simpler core. AI-native community infrastructure, filesystem-grounded, ~100× cheaper than paid-API wrappers.

If you want to build your own version, I'd suggest the build order I used:

1. **Persistence layer first.** It's the most boring move and the largest payoff. Get state out of the model's context and into version-controlled markdown. Make session-state and a write-ahead log the first files read on every boot. Everything else compounds on this.
2. **Hook layer second.** Any rule you wrote that includes "always" or "never" — port it from memory to a deterministic script. The model's attention is a shared resource; the hook layer isn't.
3. **Capture-into-primitives third.** As patterns repeat in your work (yours, the model's), save them as markdown files with trigger and action. The corpus grows monotonically and the next session inherits all of it.
4. **Anti-hallucination gates fourth.** Specifically the ones that catch claims before they ship to public or partner-facing artifacts. These are the highest-stakes failure modes.
5. **Meta-protocols last.** Once the first four layers are running, you'll see what the consistent-extension rules need to be. Don't try to write them up front.

That's the build sequence. It's the same sequence that worked for me, after a year of bouncing between layers and rediscovering that I'd skipped a foundation. Start at the bottom.

The model was amnesic. The system never was. Everything else is downstream of that.